

Urban Mobility Data Explorer

Technical Report

Team 2 | ALU Enterprise Web Development | June 2026

1. Problem Framing and Dataset Analysis

New York City is one of the most densely populated cities in the world, and understanding how people move through it is important for urban planning, transportation policy, and economic analysis. This project uses yellow taxi trip records from January 2019 provided by the NYC Taxi and Limousine Commission to build an interactive data explorer that reveals patterns in urban mobility.

The dataset contains trip-level records including pickup and dropoff timestamps, locations identified by zone IDs, passenger counts, distances, fare amounts, tips, and total charges. We integrated three data components: the trip records as a parquet file, a zone lookup CSV that maps location IDs to borough names and service zones, and spatial metadata in the form of shapefiles describing the geographic boundaries of each zone.

During the initial exploration of the data, we identified several data quality issues. A significant number of records contained missing values in required fields such as fare amount, passenger count, and trip distance. Some records had negative fare amounts or zero distances, which are physically impossible. Others had dropoff times earlier than pickup times, indicating data entry errors. Trip durations above six hours were also found, which we considered unrealistic for urban taxi trips. Additionally, some fare-to-distance ratios were extreme outliers, suggesting data corruption or fare meter errors.

One unexpected observation that influenced our design was that a large proportion of trips had a pickup location ID that did not match any entry in the zone lookup table. This meant those trips could not be assigned to a borough. We handled this by labeling them as unknown rather than excluding them, to preserve as much data as possible while maintaining transparency.

2. System Architecture and Design Decisions

Our system follows a three-tier architecture consisting of a data layer, a backend layer, and a frontend layer.

The data layer consists of a SQLite database with three tables. The zones table stores the 265 official NYC taxi zones with their borough and service zone classifications. The trips table stores cleaned and enriched trip records including three derived features. The exclusion log table records every row removed during cleaning along with the reason for exclusion, which satisfies the assignment requirement for transparency.

The backend layer is built using Python and Flask. Flask was chosen because our team had prior experience with Python from our previous MoMo SMS project, and the assignment explicitly allows it. The API exposes four GET endpoints that the frontend consumes. The data pipeline script handles loading, cleaning, feature engineering, and database insertion as a single reproducible process.

The frontend layer is a single HTML page with embedded JavaScript that fetches data from the Flask API and renders it using Chart.js. It includes summary statistic cards, a bar chart for trips by borough, a line chart for trips by hour of day, and a filterable table of individual trip records.

One key trade-off we made was choosing SQLite over PostgreSQL. SQLite requires no server setup, which made local development faster and easier for a team of five students working remotely. The trade-off is that SQLite does not scale well for production use, but for the scope of this assignment it was sufficient.

3. Algorithmic Logic and Data Structures

We implemented an insertion sort algorithm from scratch in the file `dsa/algorithm.py`. The algorithm sorts a list of trip dictionaries by fare amount without using any built-in Python sorting functions such as `sort()`, `sorted()`, or any external libraries.

The problem this solves is ranking trips by fare amount to identify the most and least expensive journeys in the dataset. This is useful for understanding pricing patterns across different boroughs and times of day.

The algorithm works as follows. Starting from the second element in the list, each trip is compared to the trips before it. If the previous trip has a higher fare amount, it is shifted one position to the right. This continues until the correct position for the current trip is found, at which point it is inserted. This process repeats for every element in the list until the entire list is sorted in ascending order.

Pseudo-code:

```
for i from 1 to length(trips):
    current = trips[i]
    j = i - 1
    while j >= 0 and trips[j].fare > current.fare:
        trips[j + 1] = trips[j]
        j = j - 1
    trips[j + 1] = current
return trips
```

Time complexity: $O(n)$ in the best case when the list is already sorted, and $O(n^2)$ in the worst case when the list is in reverse order. Average case is also $O(n^2)$.

Space complexity: $O(1)$ because the sorting is performed in place and no additional data structures are created.

We tested the algorithm on a sample of 100 trips and verified that the output was correctly ordered from the lowest fare of 2.50 dollars to the highest fare of 52.00 dollars.

4. Insights and Interpretation

Insight 1: Manhattan dominates trip origination

By querying the trips table grouped by pickup borough, we found that Manhattan accounts for approximately 90 percent of all trips in the dataset. Brooklyn is the second most common origin with around 2,000 trips, followed by Queens with approximately 1,500 trips. The Bronx and Staten Island had very few trips in comparison.

This tells us that yellow taxi services in New York City are heavily concentrated in Manhattan, which reflects the borough's high population density, tourist activity, and commercial activity. Outer boroughs rely more on other transportation options such as ride-hailing apps and public transit.

Insight 2: Late night trips have higher average fares

By grouping trips by hour of pickup, we found that trips between midnight and 3am have consistently higher average fares compared to daytime trips. The average fare during these hours was above 13 dollars, while afternoon trips averaged closer to 11 dollars.

This pattern likely reflects two factors. First, late night trips tend to be longer in distance because traffic is lighter and drivers can cover more ground in the same time. Second, some fare structures include night surcharges that increase the base rate after a certain hour.

Insight 3: Vehicle speed reveals morning congestion

Using the speed in miles per hour derived feature, we found that average trip speeds drop significantly during the morning rush hour period between 7am and 9am. Trips during this window averaged lower speeds compared to the same routes taken at midday or late evening.

This confirms what urban planners already know about Manhattan traffic, but our data provides a quantifiable measure of the congestion effect. This kind of insight could be used to recommend optimal departure times or to model the economic cost of traffic delays on taxi fare income.

5. Reflection and Future Work

The main technical challenge we faced was the size of the raw dataset. The CSV file was approximately 650 megabytes, which exceeded GitHub's file size limit and also took significant time to process on local machines. We resolved this by loading only the first 50,000 rows for local development and by excluding the raw data file from the repository using a gitignore file.

A team coordination challenge was ensuring that all five members could work on the same codebase without overwriting each other's work. We used GitHub as our version control system and assigned each member a dedicated folder to minimize conflicts.

If this were a real-world product, we would make several improvements. We would deploy the backend to a cloud server such as AWS or Heroku so the dashboard is accessible without running a local Flask server. We would load the full dataset rather than a sample of 50,000 rows to make the insights more statistically reliable. We would also add user authentication so that different organizations could access their own filtered views of the data. Finally, we would integrate the spatial GeoJSON data to render an interactive map showing trip density by zone, which would make the geographic patterns much clearer to end users.